

Final Project - Topic C: SWE-bench Codebase Adoption

Adopt a Codebase, Hard Mode

Fix five real bugs from mature Python libraries, verify the patches with SWE-bench, and explain the codebase reasoning behind each fix.

I. Background

Modern software engineering increasingly involves AI agents, but agents do not remove the need to understand the code. A patch can pass a local smoke test while missing the real edge case, or an agent can propose a plausible explanation that is wrong once you inspect the surrounding subsystem. Real codebase work requires localization, testing, patch review, and judgment.

In this project, you work with five historical GitHub issues from widely used Python libraries. Each repository is frozen at the commit before the upstream fix. The released metadata gives the issue text and base commit, but not the gold patch or hidden tests. Your task is to produce a patch for each issue, verify it through the course SWE-bench route, and explain why the patch is correct or why it failed.

II. Project Overview

You will work on five pinned SWE-bench Verified instances:

#	Instance id	Library	Bug summary
1	<code>django_django-16485</code>	django	<code>floatformat('0.00', 0)</code> raises <code>ValueError: valid range for prec</code> .
2	<code>django_django-14580</code>	django	A generated migration references <code>models.Model</code> without importing <code>models</code> .
3	<code>sphinx-doc_sphinx-8595</code>	sphinx	Empty <code>__all__ = []</code> is ignored; autodoc documents all members.
4	<code>pylint-dev_pylint-7080</code>	pylint	<code>pylint --recursive=y</code> ignores <code>ignore-paths</code> .
5	<code>pytest-dev_pytest-7571</code>	pytest	<code>caplog.set_level()</code> is not restored after a test.

Several fixes are small in lines of code, but not small in reasoning. A strong submission explains where the bug lives, how the relevant subsystem works, what local evidence supports the patch, and what the AI agent got wrong or missed.

III. Correctness and Evaluation

For each task, the released metadata includes:

- repository;
- base commit;
- issue text (`problem_statement`);
- task id used by `sb-cli`.

Correctness is evaluated through the SWE-bench harness:

- **FAILTOPASS:** tests that capture the bug should pass after your patch.
- **PASSTOPASS:** tests that previously passed should still pass.

Passing hidden tests is helpful, but it is not sufficient. An unexplained passing patch receives little analysis credit. A failed patch can still be valuable if the reproduction, localization, and failure diagnosis are careful and honest.

IV. SWE-bench Package and Submission Format

The starter package provides task metadata, problem statements, public self-check guidance, and patch naming expectations. Use its `README.md` for allowed materials, clone targets, patch names, and prediction-format expectations. You are responsible for cloning repositories, running checks, and packaging predictions yourself. Local Docker evaluation is optional.

V. Source Rules and Budget

Allowed sources:

- the source code at the assigned base commit;
- project documentation available from that checkout;
- package documentation needed to understand public APIs;
- the provided issue text.

Forbidden sources:

- the SWE-bench gold patch or hidden test patch;
- the original upstream PR or fixing commit;
- web searches for the instance id plus "fix", "patch", or the exact error;
- copying another team's transcript or patch.

No GPU is needed. Follow the course cap for agent turns or API spending. Report tokens, turns, runtime, and any failures.

VI. Analysis Scope

Your analysis should show that you understand the codebase and the bug mechanism, not merely that an agent generated a patch. For each task, connect reproduction evidence, localization, patch reasoning, local checks, and `sb-cli` status into a coherent explanation.

Across all tasks, compare at least one zero-orchestration one-shot agent run with a guided run. Explain what changed in the prompt, context, tests, or review process, and why the guided run did or did not improve the outcome. Exact artifact formats are specified in the starter README.

VII. Report Requirements

The final `report.pdf` should summarize your methodology, analysis, key findings, and conclusions. It should be self-contained and written as a coherent project report rather than a collection of patch logs.

A strong report should make clear how you adopted unfamiliar codebases, how you localized each bug, why each patch is or is not correct, and what evidence supports that judgment. For this topic, the report should also reflect on agent orchestration: where the agent helped, where it failed, and what changed between one-shot and guided runs.

The report should also contain a dedicated **Difficulties and Solutions** section with at least three concrete, verifiable challenges you encountered during the project and how you addressed them. Include an AI usage declaration explaining how AI agents were used and how their outputs were verified.

VIII. Deliverables

```
repo/
|-- patches/
|   |-- <instance_id>.patch
|   `-- predictions.jsonl
|-- analysis/
|   |-- orientation.md
|   |-- root-cause.md
|   `-- analysis.md
|-- results/sb-cli/
|-- logs/
|   |-- <instance_id>-guided.md
|   |-- <instance_id>-one-shot.md
|   `-- README.md
|-- report.pdf
`-- README.md
```

The starter README describes the expected patch files, reproduction evidence, root-cause notes, and SWE-bench status artifacts.

IX. Academic Integrity and AI Usage

Using AI agents is expected. Looking up gold patches, hidden tests, or upstream fixes is not allowed. Your patch may naturally touch the same lines as the real fix, so overlap alone is not the issue. Overlap without a derivation trail is the issue. Be prepared to explain your reasoning in the oral defense.